MONASH University

ETC4500/ETC5450
Advanced R programming

Week 7: Measuring and improving performance

Outline

1 Profiling

2 Benchmarking

3 Vectorisation

4 Copy-on-modify

5 Efficient coding

6 Caching

Outline

1 Profiling

2 Benchmarking

3 Vectorisation

4 Copy-on-modify

5 Efficient coding

6 Caching

Profiling functions

- `Rprof()` : records every function call.
- `summaryRprof()` : summarises the results.
- `profvis()` : visualises the results.

Profiling

Where are the bottlenecks in your code?

```
f <- function() {
  mean(rnorm(1e7))
  g()
  h()
}
g <- function() {
  mean(rnorm(1e7))
  h()
}
h <- function() {
  mean(rnorm(1e7))
}
```

Profiling

```r
tmp <- tempfile()
Rprof(tmp, interval = 0.1)
f()
Rprof(NULL)
```

```
sample.interval=100000
"rnorm" "mean" "f"
"rnorm" "mean" "f"
"rnorm" "mean" "f"
"rnorm" "mean" "g" "f"
"rnorm" "mean" "g" "f"
"rnorm" "mean" "h" "g" "f"
"rnorm" "mean" "h" "g" "f"
```

```
"rnorm" "mean" "h" "f"
"mean.default" "mean" "h" "f"
```

Profiling

```r
summaryRprof(tmp)
```

```
$by.self
               self.time self.pct total.time total.pct
"rnorm"              0.8    88.89        0.8     88.89
"mean.default"       0.1    11.11        0.1     11.11

$by.total
               total.time total.pct self.time self.pct
"f"                   0.9    100.00       0.0     0.00
"mean"                0.9    100.00       0.0     0.00
"rnorm"               0.8     88.89       0.8    88.89
"g"                   0.4     44.44       0.0     0.00
"h"                   0.4     44.44       0.0     0.00
```

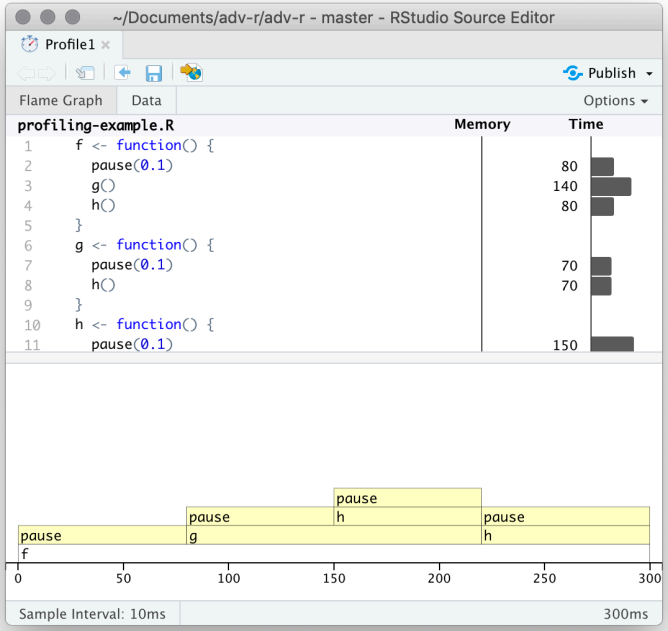
```
"mean.default"        0.1     11.11       0.1    11.11

$sample.interval
[1] 0.1

$sampling.time
[1] 0.9
```

Profiling

```
profvis::profvis(f())
```



Profiling

- Garbage collection: if it takes a long time, you're creating many short-lived objects.
- Can't see inside C/C++ code
- Anonymous functions make it harder to identify bottlenecks.
- Lazy evaluation means arguments are sometimes evaluated in a different function than where they are called.

Outline

1 Profiling

2 Benchmarking

3 Vectorisation

4 Copy-on-modify

5 Efficient coding

6 Caching

Microbenchmarking

system.time()

```
x <- rnorm(1e6)
system.time(min(x))
```

```
   user  system elapsed
  0.001   0.000   0.000
```

```
system.time(sort(x)[1])
```

```
   user  system elapsed
  0.028   0.004   0.032
```

```
system.time(x[order(x)[1]])
```

```
   user  system elapsed
  0.020   0.003   0.022
```

Microbenchmarking

bench::mark()

```r
bench::mark(
  min(x),
  sort(x)[1],
  x[order(x)[1]]
)
```

```
# A tibble: 3 × 6
  expression          min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>     <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 min(x)          766.5µs  770.7µs    1289.         0B     0
2 sort(x)[1]       28.4ms     30ms      33.1   11.44MB    15.0
3 x[order(x)[1]]   20.2ms   20.4ms      48.6    3.81MB     6.95
```

Microbenchmarking

- `mem_alloc` tells you the memory allocated in the first run.
- `n_gc` tells you the total number of garbage collections over all runs.
- `n_itr` tells you how many times the expression was evaluated.
- Pay attention to the units!

Exercises

2. What's the fastest way to compute a square root? Compare:
   - `sqrt(x)`
   - `x^0.5`
   - `exp(log(x) / 2)`

   Use `system.time()` find the time for each operation.

   Repeat using `bench::mark()`. Why are they different?

Outline

1 Profiling

2 Benchmarking

3 Vectorisation

4 Copy-on-modify

5 Efficient coding

6 Caching

Vectorisation

- Vectorisation is the process of converting a repeated operation into a vector operation.
- The loops in a vectorised function are implemented in C instead of R.
- Using `map()` or `apply()` is **not** vectorisation.
- Matrix operations are vectorised, and usually very fast.

Exercise

Write the following algorithm to estimate $\int_0^1 x^2 dx$ using vectorised code

Monte Carlo Integration

a. Initialise: `hits = 0`
b. for i in 1:N
   - Generate two random numbers, U_1, U_2, between 0 and 1
   - If $U_2 < U_1^2$, then `hits = hits + 1`
c. end for
d. Area estimate = hits/N

Exercise

Compare the speed of `apply(x, 1, sum)` and `rowSums(x)` for a large matrix `x`.

Outline

1 Profiling

2 Benchmarking

3 Vectorisation

4 Copy-on-modify

5 Efficient coding

6 Caching

Copy-on-modify semantics

- R copies objects when modified — but only when necessary.
- Names bind to the same object until one is modified.
- Understanding this explains why many inefficiencies arise.

```r
x <- c(1, 2, 3)
y <- x                          # no copy yet — x and y share memory
lobstr::obj_addr(x) == lobstr::obj_addr(y)
```

```
[1] TRUE
```

```r
y[1] <- 10                      # copy happens now
lobstr::obj_addr(x) == lobstr::obj_addr(y)
```

```
[1] FALSE
```

Why growing objects is slow

Each `c(result, i)` copies the entire vector:

```
result <- c()
for (i in 1:5) result <- c(result, i^2)
```

Pre-allocation avoids repeated copying:

```
result <- numeric(5)
for (i in 1:5) result[i] <- i^2
```

Reference semantics

- Environments use **reference semantics** — modifications don't copy.
- `data.table` exploits this: `:=` modifies in place, no copy made.

```
library(data.table)
dt <- data.table(x = rnorm(1e6))
lobstr::obj_addr(dt)
```

```
[1] "0x57a894686430"
```

```
dt[, y := x^2]              # modifies dt directly
lobstr::obj_addr(dt)        # same address — no copy
```

```
[1] "0x57a894686430"
```

Reference semantics

Compare `data.frame`:

```
df <- data.frame(x = rnorm(1e6))
lobstr::obj_addr(df)
```

```
[1] "0x57a894cc27e8"
```

```
df$y <- df$x^2                    # modifies df, but creates a copy
lobstr::obj_addr(df)              # different address — copy made
```

```
[1] "0x57a891d10418"
```

Outline

1 Profiling

2 Benchmarking

3 Vectorisation

4 Copy-on-modify

5 Efficient coding

6 Caching

Other efficiency improvements

- Never calculate something more than once.
- Avoid growing objects in a loop. Allocate space first.
- Do as little as possible.
- Move code out of loops if possible.
- Parallelise if you can't vectorise.
- Use `data.table()` instead of `data.frame()` for large datasets.
- Use `matrix()` instead of `data.frame()` or `data.table()` if all data are the same type.
- Find a built in function that does what you want instead of writing your own.

Never calculate something more than once

```r
x <- rnorm(1e4)
bench::mark(
  bad = c(mean = mean(x), var = var(x), sd = sqrt(var(x))),
  good = {
    v <- var(x)
    c(mean = mean(x), var = v, sd = sqrt(v))
  }
)
```

```
# A tibble: 2 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 bad          57.4µs   59.7µs    16539.    15.3KB     4.70
2 good         35.6µs   36.7µs    27056.        0B     2.71
```

Avoid growing objects in a loop

```r
n <- 1e4
bench::mark(
  growing = {
    result <- c()
    for (i in seq_len(n)) result <- c(result, i^2)
  },
  preallocated = {
    result <- numeric(n)
    for (i in seq_len(n)) result[i] <- i^2
  }
)
```

```
# A tibble: 2 × 6
  expression        min   median `itr/sec` mem_alloc `gc/sec`
```

```
  <bch:expr>   <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 growing        54.5ms   56.4ms      14.8     382MB    154.
2 preallocated  684.6µs  726.5µs    1252.     91.8KB     22.0
```

Do as little as possible

```r
x <- rnorm(1e6)
bench::mark(
  slow = which(x == min(x))[1],
  fast = which.min(x)
)
```

```
# A tibble: 2 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 slow         1.82ms    2.6ms      423.    7.63MB     139.
2 fast       376.71µs  379.4µs     2631.    2.09KB       0
```

Move code out of loops

```r
n <- 1e4
x <- rnorm(n)
bench::mark(
  inside = {
    result <- numeric(n)
    for (i in seq_len(n)) result[i] <- x[i] * mean(x)
  },
  outside = {
    mx <- mean(x)
    result <- numeric(n)
    for (i in seq_len(n)) result[i] <- x[i] * mx
  }
)
```

```
# A tibble: 2 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 inside        144ms    145ms      6.92    97.4KB     6.92
2 outside       834µs    883µs   1108.      94.4KB    23.9
```

Parallelise if you can't vectorise

```r
# Sequential
purrr::map(1:8, slow_fn)

# Parallel
future::plan(multisession, workers = 4)
furrr::future_map(1:8, slow_fn)
```

Use `data.table` for large datasets

```r
library(data.table)
library(dplyr)
n <- 1e6
df <- data.frame(g = sample(letters, n, replace = TRUE), x = rnorm(n))
dt <- as.data.table(df)
bench::mark(
  data.frame = summarise(df, mean(x), .by = g),
  data.table = dt[, .(mean = mean(x)), by = g],
  check = FALSE
)
```

```
# A tibble: 2 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
```

```
1 data.frame  11.01ms  11.49ms      87.0    27.3MB     251.
2 data.table   4.69ms   6.84ms     133.     27.6MB     156.
```

Use `matrix` for homogeneous data

```r
n <- 500
df <- as.data.frame(matrix(rnorm(n^2), nrow = n))
mat <- as.matrix(df)
bench::mark(
  data.frame = t(df) %*% as.matrix(df),
  matrix = t(mat) %*% mat
)
```

```
# A tibble: 2 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 data.frame   2.82ms   3.04ms      221.    7.67MB     34.4
2 matrix     718.94µs 899.72µs      988.    3.81MB     58.2
```

Outline

1 Profiling

2 Benchmarking

3 Vectorisation

4 Copy-on-modify

5 Efficient coding

6 Caching

Caching: using rds

```r
if (file.exists("results.rds")) {
  res <- readRDS("results.rds")
} else {
  res <- compute_it() # a time-consuming function
  saveRDS(res, "results.rds")
}
```

```r
res <- xfun::cache_rds(
  compute_it(), # a time-consuming function
  file = "results.rds"
)
```

Caching: using rds

```r
compute <- function(...) {
  xfun::cache_rds(rnorm(6), file = "results.rds", ...)
}
compute()
```

```
[1] -0.6894 -0.9660  0.0177 -0.4488 -0.1917  1.1685
```

```r
compute()
```

```
[1] -0.6894 -0.9660  0.0177 -0.4488 -0.1917  1.1685
```

```r
compute(rerun = TRUE)
```

```
[1]  0.2566  1.1630 -1.4097  0.0699  0.0443 -1.2448
```

```r
compute()
```

```
[1]  0.2566  1.1630 -1.4097  0.0699  0.0443 -1.2448
```

Caching downloads

Prevent downloads of the same data multiple times.

```r
download_data <- function(url) {
  dest_folder <- tempdir()
  sanitized_url <- stringr::str_replace_all(url, "/", "_")
  dest_file <- file.path(dest_folder, paste0(sanitized_url, ".rds"))
  if (file.exists(dest_file)) {
    data <- readRDS(dest_file)
  } else {
    data <- readr::read_tsv(url, show_col_types = FALSE)
    saveRDS(data, dest_file)
  }
  data
}
```

```r
bulldozers <- download_data(
  "https://robjhyndman.com/data/Bulldozers.csv"
)
```

Caching: memoise

Caching stores results of computations so they can be reused.

```r
library(memoise)
sq <- function(x) {
  cat("Computing square of 'x'")
  x^2
}
memo_sq <- memoise(sq)
memo_sq(2)
```

```
Computing square of 'x'
```

```
[1] 4
```

```r
memo_sq(2)
```

```
[1] 4
```

Exercises

Use `bench::mark()` to compare the speed of `sq()` and `memo_sq()`.

Caching: Quarto

```
```{r}
#| label: import-data
#| cache: true
d <- read.csv('my-precious.csv')
```


```{r}
#| label: analysis
#| dependson: import-data
#| cache: true
summary(d)
```
```

Caching: Quarto

- Requires explicit dependencies or changes not detected.
- Changes to functions or packages not detected.
- Good practice to frequently clear cache to avoid problems.
- targets is a better solution